

4

Building the Packtagram UI

As we leave the exciting world of chat applications behind, it's time to take on another interesting challenge – social networking. Social networking apps have seen an exponential rise in popularity over the last decade, becoming integral to our daily lives. These platforms have changed the way we communicate, share, and interact with each other on a global scale. Among them, Instagram stands out with its simplicity, its emphasis on visuals, and its engaging features, such as its newsfeed and stories.

The next few chapters are dedicated to the process of creating an Instagram-like social networking application while leveraging Android's powerful features and capabilities. We will call it Packtagram!

To start this journey, we'll begin by setting up a solid foundation and structuring our project. The structure of an Android application has a significant impact on the ease of development and the application's scalability over time. This chapter will cover various aspects of project structuring, such as defining the file hierarchy, segregating modules, and choosing the right architecture pattern for our needs.

Once our project structure is robust and scalable, we'll transition into the realm of UI development. In the case of Instagram, the primary components that catch our attention are its newsfeed and stories. We'll dive into the process of implementing these critical features, focusing on their user-friendly interfaces and seamless navigation flow.

After the UI, we'll move on to the heart of any dynamic application: data retrieval. We'll learn how to interact with servers to fetch data, focusing on the newsfeed.

In the final part of this chapter, we will venture into the world of data caching. Social media apps often involve a large amount of data transfer, and to provide a seamless and efficient user experience, effective data management strategies, including caching, are necessary. We will explore how to store stories and news items locally, thus reducing network calls and improving the app's performance.

This chapter will cover the following topics:

- Setting up Packtagram's modules and dependencies
- Creating the stories screen
- Creating the newsfeed screen and its components
- Using Retrofit and Moshi to retrieve newsfeed information
- Implementing pagination to the newsfeed

Technical requirements

As in the previous chapter, you will need to have Android Studio (or another editor of your preference) installed.

We are going to start a new project in this chapter, so it isn't necessary to download the changes that you made in the previous chapter.

Nonetheless, you can have the complete code that we are going to build through this chapter in this book's GitHub repository:

<https://github.com/PacktPublishing/Thriving-in-Android-Development-using-Kotlin/tree/main/Chapter-4>

Setting up Packtagram's modules and dependencies

To set up our app structure, we are going to create a new project. We could do this by following the same instructions that we covered in [Chapter 1](#), but we are going to introduce a variation here: our Gradle files will be written in Kotlin, and we will also use version catalogs.

Setting up a version catalog

A **version catalog** is a feature that was introduced in Gradle 7.0 to centralize the declaration of dependencies in a project. This feature provides an organized way to manage dependencies, making it easier to control and update the different versions of libraries across different modules of a project.

With a version catalog, you define all the dependencies and their versions in a **Tom's Obvious, Minimal Language (TOML)** file, usually named **libs.versions.toml**. This file resides in the Gradle folder of your project.

A version catalog offers several benefits:

- It simplifies dependency management by providing a single place to define and update the dependencies
- It minimizes errors caused by discrepancies in dependency versions across modules
- It improves the readability of build scripts by removing the need to declare each dependency individually as the declaration is centralized in a unique file

To use version catalogs, in Android Studio, fill out the details for a new project, including **Name** – here, I chose **Packtagram**. For the **Build configuration language** field, select **Kotlin DSL (build.gradle.kts)**:

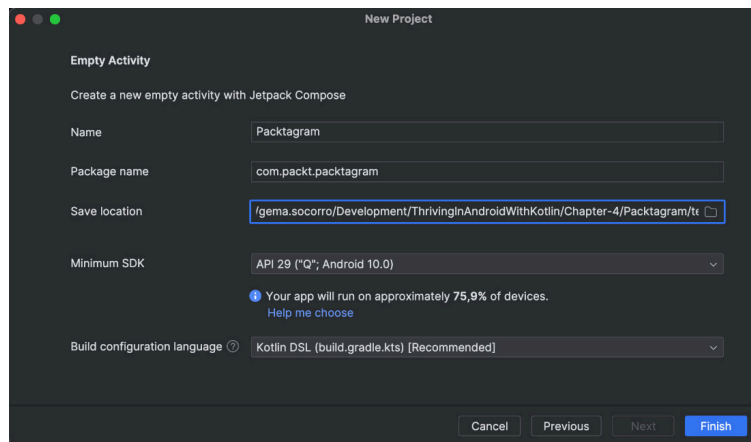


Figure 4.1: Creating a new project in Android Studio Jellyfish (2023.3.1)

With this option, Android Studio will automatically create the file needed to specify the versions. This file is called **libs.versions.toml** and its default content will look like this:

```
[versions]
agp = "8.1.0-beta01"
org-jetbrains-kotlin-android = "1.8.10"
core-ktx = "1.9.0"
...
[libraries]
core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "core-ktx" }
junit = { group = "junit", name = "junit", version.ref = "junit" }
...
[plugins]
com-android-application = { id = "com.android.application", version.ref = "agp" }
org-jetbrains-kotlin-android = { id = "org.jetbrains.kotlin.android", version.ref = "org-jetbrains" }
[bundles]
```

As shown in the following code (and in the **libs.version.toml** file generated by Android Studio in your project), the file is composed of several sections:

- **versions:** This section contains the versions of the dependencies that will be used in your project. You simply assign a reference name to each version number. This is useful to centralize versioning, particularly when the same version of a library is used in multiple places.
- **libraries:** In this block, you define your actual dependencies by assigning them an alias and linking them to the correct version defined in the **versions** block. This alias can then be used throughout your project to refer to the dependency.
- **bundles:** Bundles are groups of dependencies that are commonly used together. By creating a bundle, you can include multiple dependencies in your build scripts with a single alias. This can simplify your build scripts and make them easier to read and manage.
- **plugins:** This section is where Gradle plugins that are used in the project are defined. Similar to libraries, each plugin is given an alias and linked to a version number from the **versions** block. This feature makes managing plugins as straightforward as managing other dependencies.

Now, if we open the `gradle.build.kts` file of our app module, we'll see how the version catalog declarations are used. For example, here, we can see how the plugins are now applied:

```
plugins {  
    alias(libs.plugins.com.android.application)  
    alias(libs.plugins.org.jetbrains.kotlin.android)  
}
```

The term **alias** is used here to refer to a predefined plugin dependency that has been specified in the `libs.versions.toml` file.

Here, we can see how the dependencies are declared:

```
dependencies {  
    implementation(libs.core.ktx)  
    implementation(libs.lifecycle.runtime.ktx)  
    implementation(libs.activity.compose)  
    implementation(platform(libs.compose.bom))  
    implementation(libs.ui)  
    implementation(libs.ui.graphics)  
    implementation(libs.ui.tooling.preview)  
    implementation(libs.material3)  
    ...  
}
```

As you can see, every dependency is referred to by the name we have given them in the version catalog file (`libs.versions.toml`). It is now easier to have all the project dependencies synchronized and included in the modules.

Talking about modules, it's time we also structure our app using modularization. We already learned about the different strategies to modularize our app in [Chapter 1](#), so this is a good time to review that information.

Modularizing our app

In this case, we will segment Packtagram into several feature modules, each encapsulating distinct functionalities:

- **Newsfeed module:** The newsfeed module is dedicated to the main feed and is where users see and interact with posts from those they follow. We'll isolate this functionality because it's the core user experience and likely the first screen users will see. This module will need to handle rendering posts, managing likes and comments, and refreshing the feed.
- **Stories module:** We'll separate the stories functionality into its own module because it's a distinct user experience that requires specific UI elements and data handling. The stories module needs to manage how different user stories are rendered, track the view status, and manage story creation.
- **Profile module:** User profiles are a central part of the Instagram experience, so we'll house this functionality in the profile module. This

module will handle displaying user information, managing posts specific to a user, and editing profile details.

- **Search module:** Search functionality is complex enough to justify its own module. This module will deal with user queries, display search results, and manage interactions with search results.
- **Messaging module:** Direct messaging is a separate feature in Instagram, so we'll also isolate it in a dedicated module. This module will manage creating and displaying chats, sending and receiving messages, and notifications of new messages.
- **Core module:** This module will contain shared utilities, network interfaces, and other common components used across the application. This prevents code duplication and provides a central point for managing shared resources.

By choosing this modularization strategy, we've effectively separated our app into logical components that can be developed, tested, and debugged independently. This also aligns well with the idea of **separation of concerns**, ensuring that each part of our app has a clear, singular purpose. In the following sections, we'll explore each of these modules in detail, building the functionalities one by one, culminating in our completed social networking app.

So, let's create the modules while following the same instructions provided in [Chapter 1](#). Our module structure will look like this:

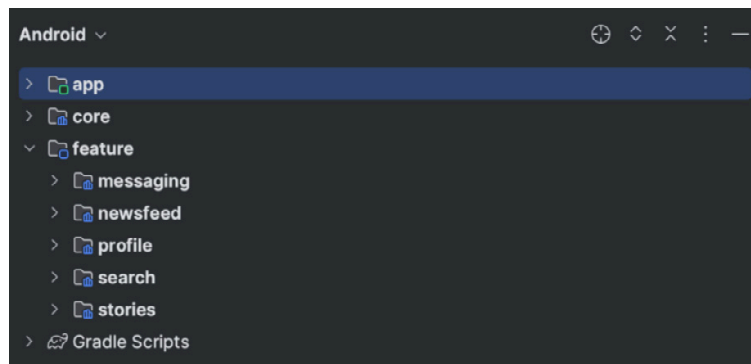


Figure 4.2: The modules structure for Packtagram

As we can see, we should have a module called **:app** (already created when creating the project), a module called **:core** for the core functionality, and a module called **:feature** that contains all of the feature modules (**:messaging**, **:newsfeed**, **:profile**, **:search**, and **:stories**).

As part of this project, we will focus on the **newsfeed** and **stories** modules (we already know how to create messaging functionality as that was covered in the last three chapters, so we don't need to cover that again).

In the case of the feature modules, we will structure them internally using the same approach we already followed in the WhatsPackt project: organizing the code and dependencies in layers. For example, we could structure the **:newsfeed** module like so:

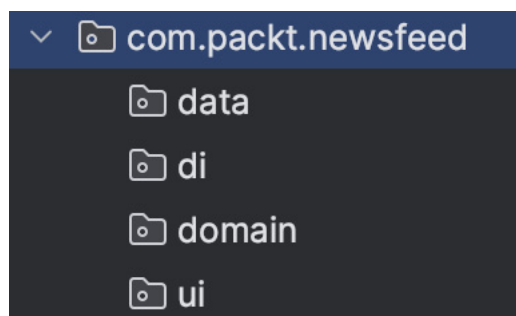


Figure 4.3: Internal structure for the feature modules

Here, we can see that we have created four internal packages:

- **data:** This is where we will place the logic for the data layer, including the components needed to retrieve the information from the backend and the data sources
- **di:** This is where we will place the logic needed to define the dependency injection instructions
- **domain:** This is where we will place the domain logic, including the repositories and use cases
- **ui:** This is where we will place all the logic related to the user interface, including ViewModels, composables, and other Android View components

We will implement the necessary components that will be part of the modules in this and the following chapters.

As part of our module structure, we've included an internal module specifically for dependency injection. Previously, in our WhatsPackt project, we used the Dagger Hilt framework for dependency injection. However, in this project, we will take a different approach by using Koin.

Getting to know Koin

Koin was mentioned briefly in [Chapter 1](#), but let's learn about its main features here:

- **Simplicity:** It offers an easy setup process and is easy to learn
- **Efficiency:** It is lightweight as it doesn't rely on reflection
- **Kotlin-centric:** Designed specifically for Kotlin, it leverages Kotlin-specific features such as extension functions, **domain-specific languages (DSLs)**, and property delegation
- **Scope management:** It has a clear way to manage the life cycle of injected instances
- **Integration:** It provides seamless integration with popular frameworks, such as ViewModel, Coroutines, and others
- **Testing:** It includes tools to simplify testing by allowing dependencies to be mocked or overridden
- **DSL configuration:** Koin uses a more readable and concise form of configuration

Let's prepare Koin for this project so that we can use it in the following sections and chapters.

Setting up Koin

To start setting up Koin, we need to add the necessary dependency to our version catalog. To do that, you will add the necessary Koin dependencies to the **libs.versions.toml** file. Be sure to use the latest version of Koin and replace **latest-version** with the actual version number:

```
[versions]
...
koin = "latest-version"
[libraries]
...
koin-core = { group = "io.insert-koin", name = "koin-core", version.ref = "koin" }
koin-android = { group = "io.insert-koin", name = "koin-android", version.ref = "koin" }
koin-androidx-navigation = { group = "io.insert-koin", name = "koin-androidx-navigation", version.ref = "koin" }
koin-androidx-compose = { group = "io.insert-koin", name = "koin-androidx-compose", version.ref = "koin" }
koin-test = { group = "io.insert-koin", name = "koin-test", version.ref = "koin" }
koin-test-junit4 = { group = "io.insert-koin", name = "koin-test-junit4", version.ref = "koin" }
```

As we can see, we have added the **koin** version in the **versions** block and the packages we might need in the **libraries** block.

Now, we need to add the dependencies to our module's Gradle files. To do this, add the following lines to the dependency Lambda:

```
dependencies {
    ...
    implementation(libs.koin.core)
    implementation(libs.koin.android)
    implementation(libs.koin.androidx.compose)
    implementation(libs.koin.androidx.navigation)
    ...
}
```

Adding these dependencies will allow us to use Koin in our modules. As a start, you should add them to the **:app**, **:feature:newsfeed**, and **:feature:stories** modules, which are the modules we are going to work with in this and the following two chapters.

Next, we need to create our **Application** class. Koin is typically initialized in your **Application** class. As we don't have one already, we will create one as part of the **:app** module and add the following code:

```
class PacktagramApplication : Application() {
    override fun onCreate() {
        super.onCreate()
        startKoin {
            androidLogger()
            androidContext(this@PacktagramApplication)
            modules(appModule)
        }
    }
}
```

In the **startKoin** block, we've specified that we want to use the Android logger (**androidLogger()**). The **androidLogger()** function is a part of Koin's API and configures Koin to use Android's native logging mechanism. Essentially, it enables Koin to print logs to Logcat.

When you initialize Koin with **androidLogger()**, you will be able to see important information about Koin's behavior and operations in Logcat while debugging your application. This includes details such as which dependencies are being created, if any errors occur during the creation of a dependency, the life cycle of scopes, and more.

After that, we provided the Android context (**androidContext(this@MyApplication)**) to the framework in case we need it to create any of our dependencies.

The next line is **modules(appModule)**. This function is where you'll list the modules that contain your project's dependencies and the instructions to provide them. To start, we will only have **appModule**, which we can create like so:

```
import org.koin.dsl.module
val appModule = module {
    ...
}
```

Inside the **module** block, we should define our dependencies once we start building them. Here's an example:

```
val exampleModule = module {
    single { MyDataSource(get()) }
    single { MyRepository(get()) }
    factory { MyUseCase(get()) }
    viewModel { MyViewModel(get()) }
}
```

The **module** function in Koin is used to define a module where you specify how to create your various dependencies. Inside a module, you can use functions such as **single**, **factory**, and **viewModel** to create instances of your dependencies.

Here's a breakdown of these functions:

- **single**: This function creates a singleton object of the specified type. Once this object is created, the same instance will be provided every time this type of object is needed. For example, **single { MyDataSource(get()) }** defines how to create a single instance of **MyDataSource**. The **get()** function inside the curly braces is a Koin function that fetches any required dependencies for creating the **MyDataSource** instance.
- **factory**: This function is used when you want to create a new instance every time the dependency is needed, instead of reusing the same in-

stance. For instance, **factory** { **MyUseCase**(get()) } creates a new **MyUseCase** object every time a **MyUseCase** instance is requested.

- **viewModel**: This function is used to create instances of **ViewModel** classes. It works like **single** but is specialized for Android's **ViewModel** instances. All **ViewModel** instances are tied to an activity or fragment life cycle and can survive configuration changes, such as screen rotation. For instance, **viewModel** { **MyViewModel**(get()) } defines how to create an instance of **MyViewModel**.
- **bind**: This function is used alongside **single**, **factory**, or **scoped** to provide additional interfaces this class can fulfill. For example, if the **MyImplementation** class implements **MyInterface**, you could enter the following:

```
single { MyImplementation() } bind MyInterface::class
```

The **get()** function that you can see in the definitions is a Koin function that automatically fetches the required dependencies. For example, if **MyDataSource** has a dependency on a **MyApi** instance, then **get()** will fetch that **MyApi** instance, provided that it has been defined somewhere in the Koin modules.

Returning to our project, we are going to leave the **appModule** empty for now – we will complete it once we start creating new components.

Talking of components, let's start with the UI we need to show the stories screen.

Creating the stories screen

In this section, we'll focus on developing a feature for creating and editing new stories within our stories feature. We'll begin by writing a **StoryEditorScreen** composable, along with its corresponding **ViewModel**, aptly named **StoryEditorViewModel**. Although this **ViewModel** will initially have limited functionality, we'll expand upon it in subsequent chapters.

Let's start creating our **ViewModel**, as follows:

```
class StoryEditorViewModel: ViewModel() {
    private val _isEditing = MutableStateFlow(true)
    val isEditing: StateFlow<Boolean> = _isEditing
}
```

In the preceding code, we are declaring **StoryEditorViewModel** and adding a property that will indicate if our screen is in edit mode or not. Edit mode will be used when the user has taken a photo or a video and wants to add more components to it.

Now, we need to take care of the dependency injection of this **ViewModel** as it must be accessible from the screen we are about to create. We can create **storyModule** in **feature:story** to be able to provide it, as follows:

```
val storyModule = module {
    viewModel<StoryEditorViewModel>()
}
```

Here, we are just telling Koin that it needs to provide **StoryEditorViewModel** where it is needed.

We also need to add this new module to the **PacktagramApplication** Koin initialization:

```
class PacktagramApplication : Application() {
    override fun onCreate() {
        super.onCreate()
        startKoin {
            androidLogger()
            androidContext(this@PacktagramApplication)
            modules(appModule, storyModule)
        }
    }
}
```

In the **modules(appModule, storyModule)** line, we have included **storyModule** to provide all the dependencies that we'll need in the stories feature.

Now, we are ready to start with the Jetpack Compose magic and create **StoryEditorScreen**. This screen will have **viewModel** as a dependency and handle **TopAppBar** and a new composable, **StoryContent**, that will hold the main functionality of the story creation and edition. We can create **StoryEditorScreen** as follows:

```
@Preview
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun StoryEditorScreen(
    viewModel: StoryEditorViewModel = koinViewModel()
) {
    val isEditing = viewModel.isEditing.collectAsState()
    Column(modifier = Modifier.fillMaxSize()) {
        if (isEditing.value) {
            TopAppBar(title = { Text(text = "Create Story") })
        }
        StoryContent(isEditing.value)
    }
}
```

As we can see, the **StoryEditorScreen** composable receives **StoryEditorViewModel** as a parameter, which provides data and functionality for this screen. This **ViewModel** is provided by Koin, using the **koinViewModel** function.

Next, **isEditing** is a state derived from the **isEditing** state flow of **ViewModel**. This state will represent whether the user is in the process of editing a story or not. The **collectAsState()** function collects the latest

value from the state flow and represents it as a state in Compose. Whenever the **isEditing** state flow emits a new value, the UI will re-compose to reflect the new state.

Inside **StoryEditorScreen**, there's a **Column** composable that takes up the maximum size of the screen. A **Column** composable allows us to arrange its children vertically. Inside this **Column**, there's a condition to check the **isEditing** state. If **isEditing** is true, **TopAppBar** will be shown with **Create Story** as its title. Here, **TopAppBar** is a composable that represents a Material Design App Bar and is generally placed at the top of the screen – this App Bar will only be shown when the user is in the editing state.

The **StoryContent** composable is then included in **Column**, outside of the condition for **isEditing**. This means that **StoryContent** will always be shown, regardless of whether the user is in editing mode or not. The **isEditing** state is passed to **StoryContent** to inform it of the current editing status. Let's work on this composable now.

This composable should have a background that will be the image or the video the user wants to include in the story and will take all the space on the screen. By doing this, the options on the screen will be different, depending on whether we're capturing the media or editing it. Here's the code for this composable:

```
@Composable
fun StoryContent(
    isEditing: Boolean = false,
    modifier: Modifier = Modifier
) {
    Box(modifier = Modifier.fillMaxSize().padding(20.dp)) {
        Box(
            modifier = Modifier
                .fillMaxWidth()
                .wrapContentHeight()
        ) {
            Button(
                onClick = { /*Handle back*/ },
                modifier =
                    Modifier.align(Alignment.TopStart)
            ) {
                Image(
                    painter = painterResource(id =
                        R.drawable.ic_arrow_back),
                    contentDescription = "Back button")
            }
            if (isEditing) {
                Button(
                    onClick =
                        { /* Handle create caption */ },
                    modifier =
                        Modifier.align(Alignment.TopEnd)
                ) {
                    Image(
                        painter = painterResource(id =
                            R.drawable.ic_caption),
                        contentDescription = "Create label"
```

```

        )
    }
}

Image(
    painter = painterResource(id =
        R.drawable.ic_default_image),
    modifier = Modifier.fillMaxSize(),
    contentDescription = "Default image"
)

Row(
    modifier = Modifier
        .wrapContentHeight()
        .align(Alignment.BottomCenter)
) {
    if (isEditing) {
        Button(
            onClick =
                { /* Handle create caption */ }
        ) {
            Text(stringResource(id =
                R.string.share_story))
        }
    } else {
        OutlinedButton(
            onClick =
                { /* Handle take a photo */ },
            modifier= Modifier.size(50.dp),
            shape = CircleShape,
            border= BorderStroke(4.dp,
                MaterialTheme.colorScheme.primary),
            contentPadding = PaddingValues(0.dp),
            colors =
                ButtonDefaults.outlinedButtonColors(
                    contentColor =
                        MaterialTheme.colorScheme.primary)
        ) {
        }
    }
}
}
}

```

Let's break down this code:

- The outermost **Box** is the main container, which takes up the maximum size of its parent and adds a padding of **20.dp**.
- The first child of **Box** is another **Box** that is set to take up the full width and wrap the content height.
- Inside this **Box**, there is a **Button** component aligned to the top-start corner of its parent **Box**. This button is used to handle a back navigation action. Inside this button is an **Image** component with an arrow icon.

NOTE

*The terms “start” and “end” are used for layout positioning to ensure better support for both **left-to-right (LTR)** and **right-to-left (RTL)** languages.*

When you use the “start” and “end” attributes in your layout, Android auto-

matically adjusts the orientation based on the text direction of the current locale. In LTR languages such as English, “start” maps to “left” and “end” maps to “right,” while in RTL languages such as Arabic, “start” maps to “right” and “end” maps to “left.” This approach simplifies the process of localizing your app for multiple languages and text directions.

- If the **isEditing** flag is true, an additional **Button** is added to **Box**. This button, which is aligned to the top end (the right, in LTR layouts) of its parent **Box**, allows users to create a caption for their story. The button uses an image of a caption icon to communicate its function.
- The next child of the outermost **Box** is **Image**, which displays a default image. This **Image** takes up the maximum size of **Box**, signifying that this image will be the main focus of this screen.
- The last child of the outermost **Box** is a **Row** that’s aligned to the bottom center of **Box**. This **Row** contains two different buttons that are displayed conditionally, based on the **isEditing** flag.
- If **isEditing** is false, **OutlinedButton** is shown. This button, styled to look like a circular button with a border, allows the user to take a photo. Note that the actual implementation for taking a photo is not included in the provided code and should be handled in the **onClick** function.
- If **isEditing** is true, a **Button** component appears instead. This button, labeled **Share Story**, is intended to allow the user to share the created story. As you can see, it is using **stringResource** with a key of **R.string.share_story**, so we should add it to **string.xml**. Again, the actual implementation of the share functionality should be handled in the **onClick** function.

With the previous code, when the screen is in editing mode, it should look like this:



Figure 4.4: The story screen in edit mode

Otherwise, when it is not in editing mode, it will look like this:



Figure 4.5: The story screen when it is not in edit mode

As we can see, it is easy and intuitive to add or remove composables conditionally from a view.

With that, we've finished the story screen, until we add more functionality in the next chapters to capture photos and video. Let's continue with the newsfeed user interface.

Creating the newsfeed screen and its components

The newsfeed is the main screen of our Packtagram app and is where the user will see the latest posts from their friends. It is structured using several components:

- **Title bar:** This is where the user can access the messaging feature
- **List of posts:** The list of posts shown in our app
- **Bottom bar:** This is used to navigate to different sections in the app

We are going to start structuring our newsfeed screen by creating a **MainScreen** composable. Here, we will define the user interface for the main view in our Packtagram app.

This **MainScreen** composable will have a **Scaffold** composable as its main component. Here, we will define the title bar and the bottom bar with the different options for navigation:

```

@Composable
fun MainScreen(
    modifier: Modifier = Modifier,
){
    val tabs = generateTabs()
    val selectedIndex = remember { mutableStateOf(0) }
    val pagerState = rememberPagerState(initialPage = 0)

```

Here, we are starting with the composable declaration and the properties we need to handle the tabs that we are going to use in the **bottomBar** navigation.

Now, it's time to add the **Scaffold** composable. This is where we will add the **title** and **bottomBar**. Let's start with the **title**:

```

Scaffold(
    topBar = {
        TopAppBar(
            title = {
                Text(stringResource(R.string.app_name))
            },
            actions = {
                IconButton(onClick =
                    { /* Menu action */ }) {
                    Icon(Icons.Rounded.Send,
                        contentDescription = "Messages")
                }
            }
        )
    },

```

With that, we have created the **Scaffold** composable and added **TopAppBar**. We used this in [Chapter 1](#), but it is important to remember that a container is generally used to hold the title of the screen and any actions relevant to the screen's context. Here, **TopAppBar** takes in two important lambdas:

- **title**: This is where you define the title of the App Bar. In this case, it's displaying a **Text** composable that fetches a string resource – that is, the name of the app (Packtagram).
- **actions**: This is where we define the actions that will appear on the right-hand side of the App Bar. Actions are typically represented with icons and are used to perform functions relevant to the current screen. In this case, there's a single **IconButton** with an envelope icon (which, when clicked, would navigate the user to the messaging screen).

The next step is to add the **BottomBar**:

```

bottomBar = {
    TabRow(selectedTabIndex = selectedIndex.value)
    {
        tabs.forEachIndexed { index, _ ->
            Tab(

```



```

        icon = { Icon(tabs[index].icon,
            contentDescription = null) },
        selected = index ==
            selectedIndex.value,
        onClick = {
            selectedIndex.value = index
        }
    )
}
}
},
},

```

Here, **BottomBar** is usually where navigation controls for the app are placed. In this case, **TabRow** is used, which is a container for **Tab** composables. The main Lambda of **TabRow** is used to generate the **Tab** elements. It iterates through each **TabItem** in **tabs** (which is a list of **TabItem** objects generated by **generateTabs()**), and for each one, it creates a **Tab** element. The **Tab** element is provided with an icon from **TabItem**, regardless of whether it's selected (based on if its index matches **selectedIndex.value**), and an **onClick** function that sets **selectedIndex.value** to the index of the clicked **Tab**.

Now, we need to add the content to the **Scaffold** composable:

```

content = { innerPadding ->
    HorizontalPager(
        modifier = Modifier.padding(innerPadding),
        pageCount = tabs.size,
        state = pagerState
    ) { index ->
        when (index) {
            0 -> {
                NewsFeed()
            }
            1 -> {
                //Search
            }
            2-> {
                // New publication
            }
            3-> {
                // Reels
            }
            4-> {
                // Profile
            }
        }
    }
}

LaunchedEffect(selectedIndex.value) {
    pagerState.animateScrollToPage(
        selectedIndex.value)
}
},

```

The **content** section is where the main content of your app goes. In this case, the content is a **HorizontalPager** composable with pages corre-

sponding to the tabs in the bottom bar.

The main Lambda in **HorizontalPager** is used to generate each page. The content of the page is determined by the index provided to the Lambda: when **index** is **0**, **NewsFeed()** is displayed, and placeholders are left for the rest of the navigation options.

There's another Lambda inside the **content** section: the **LaunchedEffect** block. This is essentially a side effect that is performed when **selectedIndex.value** changes. In this case, it triggers an animation that scrolls **HorizontalPager** to the page that corresponds to the selected index.

Now that **MainScreen** is ready, we can work on the **NewsFeed** list.

Creating the NewsFeed list

First, we need to create the **ViewModel** class that we are going to use in our **NewsFeed** composable. We will call it **NewsFeedViewModel** and add the following code:

```
class NewsFeedViewModel : ViewModel() {
    private val _posts =
        MutableStateFlow<List<Post>>(emptyList())
    val posts: StateFlow<List<Post>> get() = _posts
}
```

Here, we are initializing **NewsFeedViewModel**. For now, we will only have a public property. We'll use this to gather information so that we can render the posts in the user interface.

Now, it is time to handle the dependency injection for this **NewsFeedViewModel**. We are creating a dependency injection module per app module. So, in this case, since we are working on the newsfeed module, we will create a new dependency injection module to provide **NewsFeedViewModel**:

```
val newsFeedModule = module {
    viewModel<NewsFeedViewModel>()
}
```

Then, we will add it to the modules list in **PacktagramApplication**:

```
class PacktagramApplication : Application() {
    override fun onCreate() {
        super.onCreate()
        startKoin {
            ...
            modules(
                appModule,
                storyModule,
                newsFeedModule
            )
        }
    }
}
```

```
    }
  }
}
```

Here, we've added **newsFeedModule** to the already existing modules list in **PacktagramApplication**.

Now, we need to create the **NewsFeed** composable, which will include the list of posts:

```
@Composable
fun NewsFeed(
    modifier: Modifier = Modifier,
    viewModel: NewsFeedViewModel = koinViewModel()
) {
    LazyColumn{
        itemsIndexed(viewModel.posts){ _, post ->
            PostItem(post = post)
        }
    }
}
```

Here, we can use **LazyColumn** to render the list of posts. As we can see, we will need a **PostItem** composable to draw every list item. We will build this in the following section.

Creating the PostItem composable

Our **PostItem** composable will include all the components needed to render a post. We will need the following:

- A title bar with the picture and name of the author
- The media content (a picture initially but this could also be a video)
- An action bar with several actions (like, share, and so on)
- A label with the likes count
- A caption written by the author
- The timestamp of the post's publication

Following those requirements, this is what our **PostItem** composable will look like:

```
@Composable
fun PostItem(
    post: Post
){
    Column{
        Spacer(modifier = Modifier.height(2.dp))
        TitleBar(post = post)
        MediaContent(post = post)
        ActionBar()
        LikesCount(post = post)
        Caption(post = post)
        Spacer(modifier = Modifier.height(2.dp))
        CommentsCount(post = post)
        Spacer(modifier = Modifier.height(4.dp))
    }
}
```

```

        Timestamp(post = post)
        Spacer(modifier = Modifier.height(10.dp))
    }
}

```

As you can see, it is very readable and almost self-explanatory. We will create a **Column** composable and place every one of the composables we need vertically, leaving some spaces between as needed.

Now, let's create the composables we'll need. We will start in order, with **TitleBar**:

```

@Composable
fun TitleBar(
    modifier: Modifier = Modifier,
    post: Post
){
    Row(
        modifier = modifier
            .fillMaxWidth()
            .height(56.dp)
        ,
        verticalAlignment = Alignment.CenterVertically
    ) {
        Spacer(modifier = modifier.width(5.dp))
        Image(
            modifier = modifier
                .size(40.dp)
                .weight(1f),
            painter = painterResource(id =
                post.user.image),
            contentDescription =
                "User ${post.user.name} avatar"
        )
        Text(
            text = post.user.name,
            modifier = modifier
                .weight(8f)
                .padding(start = 10.dp),
            fontWeight = FontWeight.Bold
        )
        IconButton(onClick = { /* Menu options */}) {
            Icon(
                Icons.Outlined.MoreVert,
                "More options"
            )
        }
    }
}

```

The base for this composable will be a **Row** composable as it will arrange its children in a horizontal sequence. The **verticalAlignment** parameter is set to **Alignment.CenterVertically** to align the items in the row vertically in the center.

Here's a description of the children composables that were used:

- **Spacer**: This is used to provide some space on the interface. Here, it provides a width of **5.dp** at the start of the row.
- **Image**: This is an image composable that is being used to display the user's profile picture. The image source is taken from the **Post** object that was passed in.
- **Text**: This displays the user's name and takes the name from the **Post** object that was passed in. The **fontWeight** parameter is set to **FontWeight.Bold** to make the text bold.
- **IconButton**: This is a button with an icon. The **onClick** parameter is a Lambda function that gets called when the button is clicked. In this case, the function is empty, but this is where you would put code to handle the button press. The **Icon** element inside is used to display the **more-options** icon.

Now that **TitleBar** is ready, it's time for the **MediaContent** composable, which will display the content that the user has posted:

```
@Composable
fun MediaContent (
    modifier: Modifier = Modifier,
    post: Post
){
    Box(
        modifier = modifier
            .fillMaxWidth()
            .height(300.dp),
        contentAlignment = Alignment.Center,
    ) {
        Image(
            modifier = Modifier
                .fillMaxSize(),
            painter = rememberImagePainter(post.image),
            contentDescription = null
        )
    }
}
```

The preceding code generates a box that contains an image as this composable is used to display the main image content of a post. These are the main components:

- **Box**: This is a layout composable that stacks its children. In this case, it's used to hold an **Image** component. The **contentAlignment** parameter is set to **Alignment.Center** to center the image in the box and has a modifier applied to it to fill the maximum width and to set a height of **300.dp**.
- **Image**: This is an **Image** composable that's used to display an image. The image source is taken from the **Post** object passed in. The modifier is used to ensure the image fills the maximum size of the **Box** composable. Here, **rememberImagePainter** is used to load and display the image from a source (such as a URL or a local file), and it's remembered across recompositions.

Now that we've completed the **MediaContent** composable, we will consider **ActionBar**, which will provide the instructions to render the action buttons:

```
@Composable
fun ActionBar(
    modifier: Modifier = Modifier,
){
    Column(
        modifier = modifier
            .fillMaxWidth()
            .height(40.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
    ) {
        Row(
            modifier = modifier
                .fillMaxSize()
        ) {
            Row(
                modifier = modifier
                    .fillMaxHeight()
                    .weight(1f)
                ,
                verticalAlignment =
                    Alignment.CenterVertically,
            ) {
                IconButton(onClick = { }) {
                    Icon(
                        imageVector =
                            Icons.Outlined.Favorite,
                        contentDescription = "like",
                        modifier = modifier
                    )
                }
                IconButton(onClick = { }) {
                    Icon(
                        imageVector = Icons.Outlined.Edit,
                        contentDescription = "comment",
                        modifier = modifier
                    )
                }
                IconButton(onClick = { }) {
                    Icon(
                        imageVector = Icons.Outlined.Share,
                        contentDescription = "share",
                        modifier = modifier
                    )
                }
            }
            Row(
                modifier = modifier
                    .fillMaxHeight()
                    .weight(1f)
            ) {
            }
            Row(
                modifier = modifier
                    .fillMaxHeight()
                    .weight(1f),
                verticalAlignment =
```

```

        Alignment.CenterVertically,
        horizontalArrangement = Arrangement.End
    ) {
        IconButton(onClick = { }) {
            Icon(
                imageVector =
                    Icons.Outlined.Star,
                contentDescription =
                    "bookmark",
            )
        }
    }
}
}
}
}
}
}
}
}
}
}

```

The preceding code generates a UI that represents the action buttons under a post, similar to those on Instagram where you can like, comment, share, and bookmark a post. Here's what each part of the function does:

- **Column:** This creates a column in which you can place other UI elements vertically. The **horizontalAlignment** parameter is set to **Alignment.CenterHorizontally**, which centers the elements horizontally in the column.
- **Row:** This creates a row in which other UI elements can be placed horizontally. It fills the maximum size of the parent, which is **Column**.
- The first group of three **IconButton** composables are in a **Row** composable and are for the **Like**, **Comment**, and **Share** actions. Each **IconButton** takes a Lambda for the **onClick** event, which currently does nothing.
- There are then two additional **Row** composables, both with **fillMaxHeight().weight(1f)**, that appear to be placeholders, perhaps for adding additional icons in the future.
- The final **Row** composable has an **IconButton** composable for the **Bookmark** action. It has **verticalAlignment** set to **Alignment.CenterVertically** and **horizontalArrangement** set to **Arrangement.End** to position the icon in the center vertically and at the end (right-hand side in LTR layouts) of the available space horizontally.
- **Icon:** Each **Icon** displays an image and has a **contentDescription** composable for accessibility purposes. The **modifier** parameter can be used to adjust the layout or other visual properties of the icon.

Having configured the **ActionBar** composable to provide a flexible UI layout featuring a range of interactive buttons, our next focus will be the likes count. It's very straightforward to implement:

```

@Composable
fun LikesCount(
    modifier: Modifier = Modifier,
    post: Post
){
    Row(

```

```

        modifier = modifier
            .fillMaxWidth()
            .height(30.dp)
            .padding(horizontal = 10.dp)
        ,
        verticalAlignment = Alignment.CenterVertically
    ) {
        Text(
            text = post.likesCount.toString().plus(
                «likes»),
            fontWeight = FontWeight.Bold,
            fontSize = 16.sp
        )
    }
}

```

The **LikesCount** function is a **Composable** function that creates a row to display the number of likes that a post has received. Here's what each part of the function does:

- **Row:** This creates a row in which other UI elements can be placed horizontally. It uses the provided modifier to fill the maximum width of the parent container, setting its height to **30.dp** and adding a padding of **10.dp** horizontally. The **verticalAlignment** parameter is set to **Alignment.CenterVertically**, which centers the elements vertically in the row.
- **Text:** This creates a text element that displays the number of likes that the post has received. It gets the **likesCount** field from the **Post** object, converts it into a string, and appends the word **likes** to the end. It also sets the font weight to bold and the font size to **16.sp**.

The next composable is the caption, which is the text that the user adds to the post:

```

@Composable
fun Caption(
    modifier: Modifier = Modifier,
    post: Post
){
    Row(
        modifier = modifier
            .fillMaxWidth()
            .wrapContentHeight()
            .padding(horizontal = 10.dp)
        ,
        verticalAlignment = Alignment.CenterVertically
    ) {
        Text(
            text = buildAnnotatedString {
                val boldStyle = SpanStyle(
                    fontWeight = Bold,
                    fontSize = 14.sp
                )
                val normalStyle = SpanStyle(
                    fontWeight = FontWeight.Normal,
                    fontSize = 14.sp
                )
            }
        )
    }
}

```



```

        pushStyle(boldStyle)
        append(post.user.name)
        append(" ")
        if (post.caption.isNotEmpty()){
            pushStyle(normalStyle)
            append(post.caption)
        }
    }
}
)
}
}

```

Here's what each part of the function does:

- **Row:** This creates a row in which other UI elements can be placed horizontally. It uses the provided modifier to fill the maximum width of the parent container, wrap its height to its content, and add a padding of **10.dp** horizontally. The **verticalAlignment** parameter is set to **Alignment.CenterVertically**, which centers the elements vertically in the row.
- **Text:** This creates a text element that displays the caption of the post, preceded by the user's name. The **buildAnnotatedString** function is used to build a string with different text styles for different parts. Thanks to that, the user's name is styled with a bold font weight, and the caption is styled with a normal font weight.

Upon completing the **Caption** composable, let's tackle the **CommentsCount** composable:

```

@Composable
fun CommentsCount(
    modifier: Modifier = Modifier,
    post: Post
) {
    Row(
        modifier = modifier
            .fillMaxWidth()
            .wrapContentHeight()
            .padding(horizontal = 10.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Text(
            text = stringResource(R.string.comment_count,
                post.commentsCount),
            fontWeight = FontWeight.Normal,
            fontSize = 14.sp
        )
    }
}

```

The **CommentsCount** composable creates a layout to display the number of comments on a post. Here's what each part of the function does:

- **Row:** This creates a row in which other UI elements can be placed horizontally. It uses the provided modifier to fill the maximum width of the parent container, wrap its height to its content, and add a padding of **10.dp** horizontally. The **verticalAlignment** parameter is set to

Alignment.CenterVertically, which centers the elements vertically in the row.

- **Text:** This creates a text element that displays the number of comments. The **stringResource** function is used to get a string resource, which is a format string that takes a number and inserts it into the correct place to form a string that says **Read 3 comments**. The string format is then filled in with the number of comments from the **Post** object.

Now that we've finished implementing the **CommentsCount** composable, we will create the **TimeStamp** composable:

```
fun TimeStamp(
    modifier: Modifier = Modifier,
    post: Post
) {
    Row(
        modifier = modifier
            .fillMaxWidth()
            .wrapContentHeight()
            .padding(horizontal = 10.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Text(
            text = "${post.timeStamp} hours ago ",
            fontSize = 10.sp,
            fontWeight = FontWeight.Light
        )
    }
}
```

The **TimeStamp** function is a composable that creates a layout to display the timestamp of a post. Here's what each part of the function does:

- **Row:** This creates a row in which other UI elements can be placed horizontally. It uses the provided **Modifier** value to fill the maximum width of the parent container, wrap its height to its content, and add a padding of **10.dp** horizontally. The **verticalAlignment** parameter is set to **Alignment.CenterVertically**, centering the elements vertically in the row.
- **Text:** This creates a text element that displays the timestamp. The **text** parameter of this function is set to a string that includes the **timeStamp** attribute from the **Post** object. The **fontSize** and **fontWeight** parameters set the size and weight of the font to **10.sp** and **FontWeight.Light**, respectively.

With this composable, we have finished the **Post** composable components. If we do a preview with fake data, we'll see the following screen:

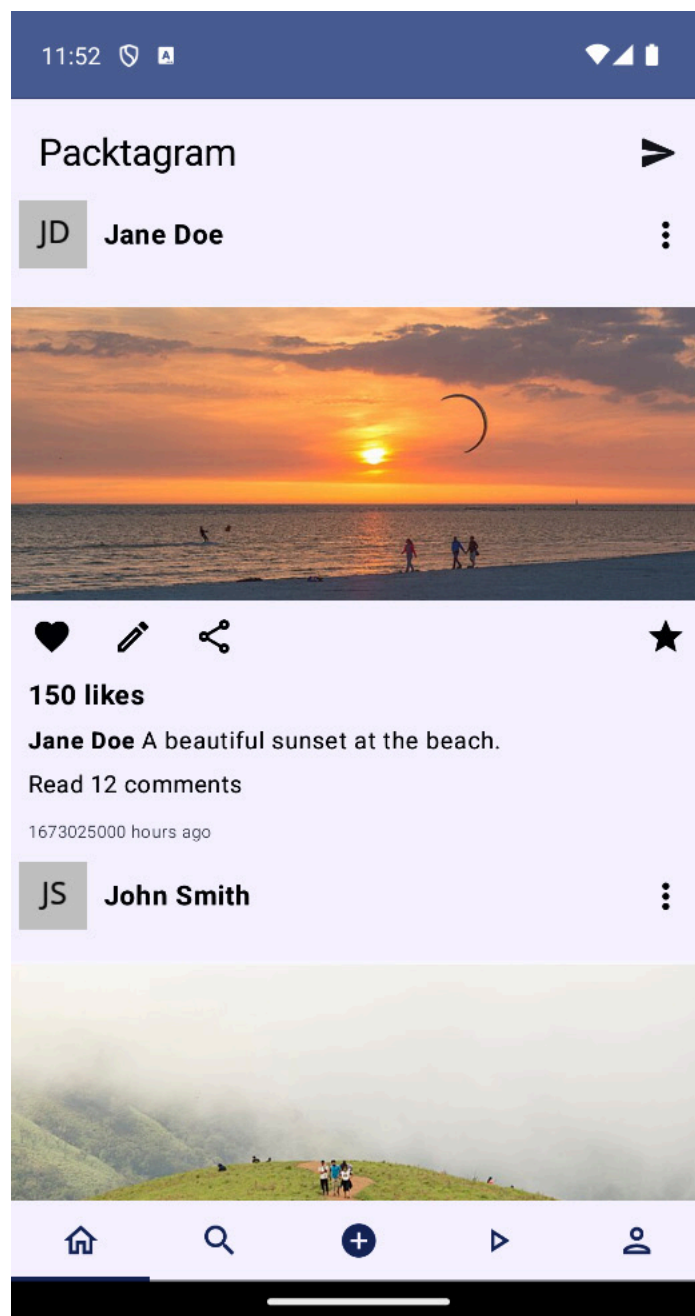


Figure 4.6: Newsfeed screen

Now, it is time to implement a way to populate the information needed from the backend.

Using Retrofit and Moshi to retrieve newsfeed information

In this section, we're going to prepare our app so that it can retrieve the newsfeed information from the backend. To do that, we'll need to create an HTTP client that handles the calls to the backend services. Since we used **ktor** in our first project, we are going to take a different approach for this one and use **Retrofit**.

Retrofit is a type-safe HTTP client for Android and Java (which is completely compatible with Kotlin). Retrofit makes it easy to connect to a

REST web service by translating the API into Kotlin or Java interfaces.

Here are some of its main features:

- **Easy to use:** Retrofit turns your HTTP API into a Kotlin or Java interface. All you have to do is define the API's URL and method (**GET**, **POST**, and so on) using annotations. Retrofit will automatically convert the HTTP responses into data objects.
- **Type conversion:** By default, Retrofit can only deserialize HTTP bodies into OkHttp's **ResponseBody** type and it can only accept its **RequestBody** type for **@Body**. Converters can be added to support other types. For example, a JSON converter can be used to convert the API's responses into Kotlin or Java objects automatically.
- **HTTP methods annotations:** You can use annotations to describe HTTP methods such as **GET**, **POST**, **DELETE**, **UPDATE**, and others. You can also use other annotations, such as **Headers**, **Body**, **Field**, **Path**, and more, to make your request exactly as needed.
- **URL parameter replacement and query parameter support:** Add parameters to your request with annotations. For example, you can add a path parameter by setting the specific value in the URL, or you can add a query parameter at the end of the URL.
- **Synchronous and asynchronous calls:** Retrofit supports both synchronous (blocking) calls and asynchronous (non-blocking) calls. For Android, asynchronous calls are more important as network operations on the main thread are discouraged.
- **Support for coroutines and RxJava:** Retrofit provides out-of-the-box support for coroutines and RxJava. This makes it easy to use with these popular libraries for handling asynchronous operations.
- **Interceptors:** Retrofit also allows you to use OkHttp's interceptors. You can add headers to every request or log the request and response data for debugging purposes.

We also need to use a converter to parse the backend responses into objects. We will use **Moshi** (<https://github.com/square/moshi>) for that. Moshi is a modern JSON library for Android and Java, also built by Square. It aims to be easy to use and efficient, and its design is inspired by the well-regarded Gson library, but it seeks to improve upon several design aspects. Here are some of its main features:

- **Easy to use:** Moshi provides simple **toJson()** and **fromJson()** methods to convert Java and Kotlin objects into JSON and vice versa.
- **Built-in and custom converters:** Moshi has built-in support for converting many common types and can encode any object graph of these. For other classes, you can write custom converters, called **adapters**, that define how these types are converted to and from JSON.
- **Kotlin support:** Moshi supports Kotlin and provides the **moshi-kotlin-codegen** module, which leverages annotation processing to generate adapters for your Kotlin classes automatically.
- **Null safety:** Moshi handles **null** values in the JSON input and can be configured to allow or disallow **null** values in your Java or Kotlin objects.

- **Annotation-based:** Much like Retrofit, Moshi uses annotations to denote special behavior for certain fields (for example, custom names and transient values).
- **Fault-tolerant:** Moshi is fault-tolerant and will not fail the entire operation if it encounters unknown properties or incompatible types in the JSON data. This can be beneficial when dealing with APIs that may occasionally change.
- **Efficient:** Moshi is designed to be efficient in its operation, minimizing object allocation and garbage collection overhead.

Now that we know the advantages of using Retrofit with Moshi, let's start integrating them into our project.

Adding the Retrofit and Moshi dependencies

To use both the Retrofit and Moshi libraries, we need to configure their dependencies. First, we will add them to the versions catalog file:

```
[versions]
...
retrofit = "2.9.0"
moshi = "1.12.0"
coroutines = "1.5.1"
moshi-converter = "0.8.0"
...
[libraries]
...
retrofit = { group = "com.squareup.retrofit2", name = "retrofit", version.ref="retrofit"}
retrofitMoshiConverter = { group = "com.squareup.retrofit2", name = "converter-moshi", version.ref = "moshi" }
moshi = { group = "com.squareup.moshi", name = "moshi", version.ref = "moshi" }
moshiKotlin = { group = "com.squareup.moshi", name = "moshi-kotlin", version.ref = "moshi" }
moshiKotlinCodegen = { group = "com.squareup.moshi", name = "moshi-kotlin-codegen", version.ref = "moshi" }
retrofitKotlinxSerialization = { group = "com.squareup.retrofit2", name = "retrofit-kotlinx-serialization-converter", version.ref = "retrofit" }
coroutinesCore = { group = "org.jetbrains.kotlinx", name = "kotlinx-coroutines-core", version.ref = "coroutines" }
coroutinesAndroid = { group = "org.jetbrains.kotlinx", name = "kotlinx-coroutines-android", version.ref = "coroutines" }
...
kotlin-kapt = { id = "org.jetbrains.kotlin.kapt", version.ref = "org-jetbrains-kotlin-android" }
```

Then, we will include the dependencies in our module's **build.gradle.kts** file, making them available to use in our module:

```
dependencies {
    implementation(libs.retrofit)
    implementation(libs.retrofitMoshiConverter)
    implementation(libs.moshiConverter)
    implementation(libs.moshi)
    implementation(libs.moshiKotlin)
    kapt(libs.moshiKotlinCodegen)
    implementation(libs.coroutinesCore)
    implementation(libs.coroutinesAndroid)
    ...
}
```

After adding these dependencies, we should be ready to work with both libraries. Let's start creating our data source so that we can obtain the data for the newsfeed.

Creating the data source for the newsfeed

At this point, we're ready to create our data source. We will do this in the `:feature:newsfeed` module. First, we need to create an interface to define our API endpoints using Retrofit. We can use `@GET`, `@POST`, and others to define what kind of HTTP request we want to make:

```
interface NewsFeedService {
    @GET("feed")
    suspend fun getNewsFeed(): List<PostApiData>
}
```

This is an interface for the Retrofit library that's used to turn the HTTP API into a Kotlin interface. It also defines an endpoint for your API:

- **interface NewsFeedService:** This is declaring a new interface named **NewsFeedService**.
- **@GET("feed"):** This is an annotation that describes an HTTP GET request. The **"feed"** parameter is the endpoint where the request will be sent. So, the full URL for this request would be something like <https://packtagram.com/feed> if the base URL of your Retrofit client is <https://packtagram.com/>.
- **suspend fun getNewsFeed(): List<PostApiData>:** This is declaring a function named **getNewsFeed** that is expected to return a list of **Post** objects. The **suspend** keyword means that this function is a suspending function, which is a type of function that can be paused and resumed at a later time. This will be called later from a coroutine.

So, to put it all together, when **getFeed** is called, it will make a **GET** request to the <https://packtagram.com/feed> URL and expect to receive a JSON array of **PostApiData** objects, which are then parsed into a list of **Post** objects in your Kotlin code.

To see an example of a JSON file that contains the expected fields, check out <https://api.mockfly.dev/mocks/09e4e43e-7992-4dd7-b99f-e168667a240e/feed>.

Now, we need to generate a client from this interface. For that, we will use the Retrofit builder:

```
object RetrofitInstance {
    private const val BASE_URL = "https://packtagram.com/"
    fun getNewsFeedApi(): NewsFeedService = run {
        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(
                MoshiConverterFactory.create()
            )
            .build()
    }
}
```

```
        .create(NewsFeedService::class.java)
    }
}
```

Here, we are creating a function called **getNewsFeedApi()** that will build the **NewsFeedService** client. For that, we need a **BASE_URL** function that we can hardcode in this same file. The recommendation is to store this information in a configuration file so that we can easily change it if we need to have different **buildTypes** of the app, for example.

We are also adding the Moshi converter using the **.addConverterFactory(MoshiConverterFactory.create())** function. This will allow Retrofit to deserialize the backend responses using Moshi.

Now, we need to create **NewsFeedRemoteDataSource**:

```
class NewsFeedRemoteDataSource(private val api:
NewsFeedService) {
    suspend fun getNewsFeed(): List<PostApiData> {
        return api.getNewsFeed()
    }
}
```

As we can see, we have created a **NewsFeedRemoteDataSource** composable. Here, we will have one function, **getNewsFeed()**. This function will call **NewsFeedService** to obtain the newsfeed.

Now, let's create the repository for this newsfeed.

Creating the repository

The next step is to define the repository that will orchestrate the information gathering and storage using the different data sources (for now, we only have one, **NewsFeedRemoteDataSource**). It will also map the information into the new layer: the domain layer.

First, we'll define its interface as part of the domain layer:

```
interface NewsFeedRepository {
    suspend fun getNewsFeed(): List<Post>
}
```

Second, we'll implement its functionality as part of the data layer:

```
class NewsFeedRepositoryImpl(
    private val remoteDataSource: NewsFeedRemoteDataSource
): NewsFeedRepository {
    override suspend fun getNewsFeed(): List<Post> {
        return remoteDataSource
            .getNewsFeed()
            .map { it.toDomain() }
    }
}
```

As we can see, for now, it will only have a `getNewsFeed()` function that will obtain a list of **Post** objects. It will obtain the newsfeed from the remote data source and map the **PostApiData** objects into **Post** objects.

Now, let's create the use case to obtain this data.

Creating the GetTheNewsFeedUseCase

As we progress through the layers, the next step will be to create the use cases needed. In this case, we will create a use case to obtain the newsfeed – that is, **GetTheNewsFeedUseCase**:

```
class GetTheNewsFeedUseCase(  
    private val repository: NewsFeedRepository  
) {  
    suspend operator fun invoke(): List<Post> {  
        return repository.getNewsFeed()  
    }  
}
```

Here, we are using **repository** to get the newsfeed in the **invoke** function.

Before continuing, we need to create the data classes that we will be using in the data and domain layers. In the case of the domain layer, we will create the **Post** data class:

```
data class Post(  
    val id: String,  
    val user: UserData,  
    val imageUrl: String,  
    val caption: String,  
    val likesCount: Int,  
    val commentsCount: Int,  
    val timeStamp: Long  
) {  
    data class UserData(  
        val id: String,  
        val name: String,  
        val imageUrl: String  
    )  
}
```

Here, we are declaring all the fields that we need for the **Post** object in the domain layer.

In the case of the data layer, we will create the **PostApiData** data class and mapping functions that we'll need to map to the domain object:

```
data class PostApiData(  
    @Json(name = "id")  
    val id: String,  
    @Json(name = "author")  
    val user: UserApiData,
```



```

@Json(name = "image_url")
val imageUrl: String,
@Json(name = "caption")
val caption: String,
@Json(name = "likes_count")
val likesCount: Int,
@Json(name = "comments_count")
val commentsCount: Int,
@Json(name = "timestamp")
val timeStamp: Long
) {
    data class UserApiData(
        @Json(name = "id")
        val id: String,
        @Json(name = "name")
        val name: String,
        @Json(name = "image_url")
        val imageUrl: String
    ) {
        fun toDomain(): Post.UserData {
            return Post.UserData(
                id = id,
                name = name,
                imageUrl = imageUrl
            )
        }
    }
    fun toDomain(): Post {
        return Post(
            id = id,
            user = user.toDomain(),
            imageUrl = imageUrl,
            caption = caption,
            likesCount = likesCount,
            commentsCount = commentsCount,
            timeStamp = timeStamp
        )
    }
}

```

Here, we should include the fields that the response from the backend will return to our app. Note that we are using the `@Json(name = "")` annotation in the properties to specify the name of the field in the JSON that the backend will return.

Before jumping to consume the use case in **ViewModel**, we have to sort out the dependency injection for all the components we have just created. We will do so in **newsFeedModule**:

```

val newsFeedModule = module {
    single { RetrofitInstance.getNewsFeedApi() }
    single { NewsFeedRemoteDataSource(get()) }
    single<NewsFeedRepository> {
        NewsFeedRepositoryImpl(get()) }
    factory { GetTheNewsFeedUseCase(get()) }
    viewModel<NewsFeedViewModel>()
}

```

Now, it is time to integrate this use case into **NewsFeedViewModel**.

Integrating the use case into our ViewModel

For the ViewModel, we will need to create a new function to obtain the posts. We will call it `loadPosts()`:

```
init {
    loadPosts()
}
private fun loadPosts() {
    viewModelScope.launch {
        val newPosts = getTheNewsFeedUseCase()
        _posts.value = newPosts
    }
}
```

Here, we are loading the posts as soon as the app shows the view and the ViewModel is created.

The changes to the `posts` property are already being consumed by our `NewsFeed` composable, so it will update the UI when it receives any posts.

We could also add some error handling here, but as we already worked on that topic in the first project, we will leave it here.

Now, it wouldn't be realistic (and performant) to load all the existing posts initially. As we did with the messaging project, we'll need to paginate so that we can obtain the posts gradually, following the user scroll. We will see how in the next section.

Implementing pagination in the newsfeed

To implement pagination in our app, we will start by modifying the Retrofit service. Typically you're required to add parameters to your API endpoint that control the "page" of data you're requesting. For instance, we might have a `pageNumber` parameter and a `pageSize` parameter (though this will depend on the design of your backend endpoints).

First, let's adjust `NewsFeedService` so that it includes the two parameters we just mentioned:

```
interface NewsFeedService {
    @GET("/feed")
    suspend fun getNewsFeed(
        @Query("pageNumber") pageNumber: Int,
        @Query("pageSize") pageSize: Int
    ): List<PostApiData>
}
```

Now, we will need to change the signature of the data source function so that it includes those fields. In the data source, we will change the follow-

ing function:

```
suspend fun getNewsFeed(pageNumber: Int, pageSize:
Int): List<PostApiData> {
    return api.getNewsFeed(pageNumber, pageSize)
}
```

In the repository, we will handle storing the current page and keeping the desired size of pages (this could also be a constant somewhere):

```
class NewsFeedRepositoryImpl(
    private val remoteDataSource:
    NewsFeedRemoteDataSource): NewsFeedRepository
{
    private var currentPage = 0
    private val pageSize = 20 // Or whatever page size we
    prefer
    override suspend fun getNewsFeed(): List<Post> {
        return remoteDataSource
            .getNewsFeed(currentPage, pageSize)
            .map { it.toDomain() }
            .also { currentPage++ }
    }
    fun resetPagination() {
        currentPage = 0
    }
}
```

Here, we are storing the current page so that when we call the data source, we can specify if we want the next one. We have also added a function called **resetPagination()** that will reset the current page so that we can start again.

Next, we are going to use **resetPagination()** in **UseCase** when the user navigates to the top and wants to get the first of the publications:

```
suspend operator fun invoke(fromTheBeginning: Boolean):
List<Post> {
    if (fromTheBeginning) {
        repository.resetPagination()
    }
    return repository.getNewsFeed()
}
```

The next step is to handle when we should load the next page and load the next posts. To do so, we'll need to modify our **NewsFeed** composable and **NewsFeedViewModel**.

First, we are going to implement the **NewsFeedViewModel** part:

```
init {
    loadInitialPosts()
}
private fun loadInitialPosts() {
```

```
viewModelScope.launch {
    val newPosts = withContext(dispatcher) {
        getTheNewsFeedUseCase(fromTheBeginning =
            true)
    }
    _posts.value = newPosts
}
fun loadMorePosts() {
    viewModelScope.launch {
        val newPosts = withContext(dispatcher) {
            getTheNewsFeedUseCase(fromTheBeginning =
                false)
        }
        val updatedPosts = (_posts.value +
            newPosts).takeLast(60)
        _posts.value = updatedPosts
    }
}
```

Here, we've renamed our initial function **loadInitialPosts()** so that it indicates that it will load the first posts. Then, we created a new function called **loadMorePosts()** that will load the new page. It will add it to the existing post list.

Now, we need to make a few modifications to our **NewsFeed** composable so that it will call the **ViewModel** when a new page is needed. For that reason, we need to create a **LazyListState** extension that we will invoke whenever the user reaches the end of the list:

```
fun LazyListState.OnBottomReached(
    loadMore : () -> Unit
){
    val shouldLoadMore = remember {
        derivedStateOf {
            val lastItemInView =
                layoutInfo.visibleItemsInfo.lastOrNull()
                ?: return@derivedStateOf true
            lastItemInView.index ==
                layoutInfo.totalItemsCount - 1
        }
    }
    LaunchedEffect(shouldLoadMore){
        snapshotFlow { shouldLoadMore.value }
        .collect {
            if (it) loadMore()
        }
    }
}
```

This extension function observes the scrolling state of **LazyColumn** or **LazyRow**. When the user has scrolled to the bottom, it calls the provided **loadMore** function to load more items. This pattern is common in implementing “infinite scrolling” or “pagination,” which is what we are currently implementing.

Now, we need to use it in our **LazyColumn** layout. For that, we need to remember **LazyListState** in the **NewsFeed** composable:

```
@Composable
fun NewsFeed(
    modifier: Modifier = Modifier,
    viewModel: NewsFeedViewModel = koinViewModel()
) {
    val posts = viewModel.posts.collectAsState()
    val listState = rememberLazyListState()
    LazyColumn{
        itemsIndexed(posts){ _, post ->
            PostItem(post = post)
        }
    }
    listState.OnBottomReached {
        viewModel.loadMorePosts()
    }
}
```

With this change, every time our user reaches the bottom of the list, we will call the function to load more posts and get the next page.

Now that we've finished our paging implementation, our user experience will be more performant and smoother when navigating the newsfeed.

Summary

In this chapter, we primarily focused on structuring and modularizing our Packtagram app while enhancing maintainability. Leveraging Jetpack Compose, we designed the components and screens for some of the features of the interface that we are going to be working on in the next chapters.

Additionally, this chapter delved into the intricacies of connecting the developed UI to the backend, which is pivotal for data management and operation handling. We implemented Retrofit for network operations and Moshi for JSON parsing, bridging the gap between the user interface and the data source. Moreover, we introduced the concept of paging to efficiently manage large datasets. By doing so, we ensured smoother data load, faster response times, and enhanced app performance overall, significantly improving the user's experience.

In the next chapter, we will dive into the photo functionality of our app. We will use an incredible library called CameraX and take advantage of some of its capabilities. We will also learn how to apply machine learning to our camera preview using ML Kit.